

TDS200 · Kryssplattform

Session 4

Forms, Modals & Firebase

Design · Controlled Input · Validation · Modal · Comments · Firebase

Cross-Platform Mobile Development · Lecturer: Yuan Lin

What you'll learn today

- Theory: Designing a React Native app
- Build controlled inputs with validation
- Use the Modal component for create/confirm flows
- Add comments to a post
- What Backend-as-a-Service (BaaS) means and when to use it
- An introduction of Firebase
- First Firebase project setup



Modal Popup

A modal 'New post' form + a comment box on the detail screen



Firebase step

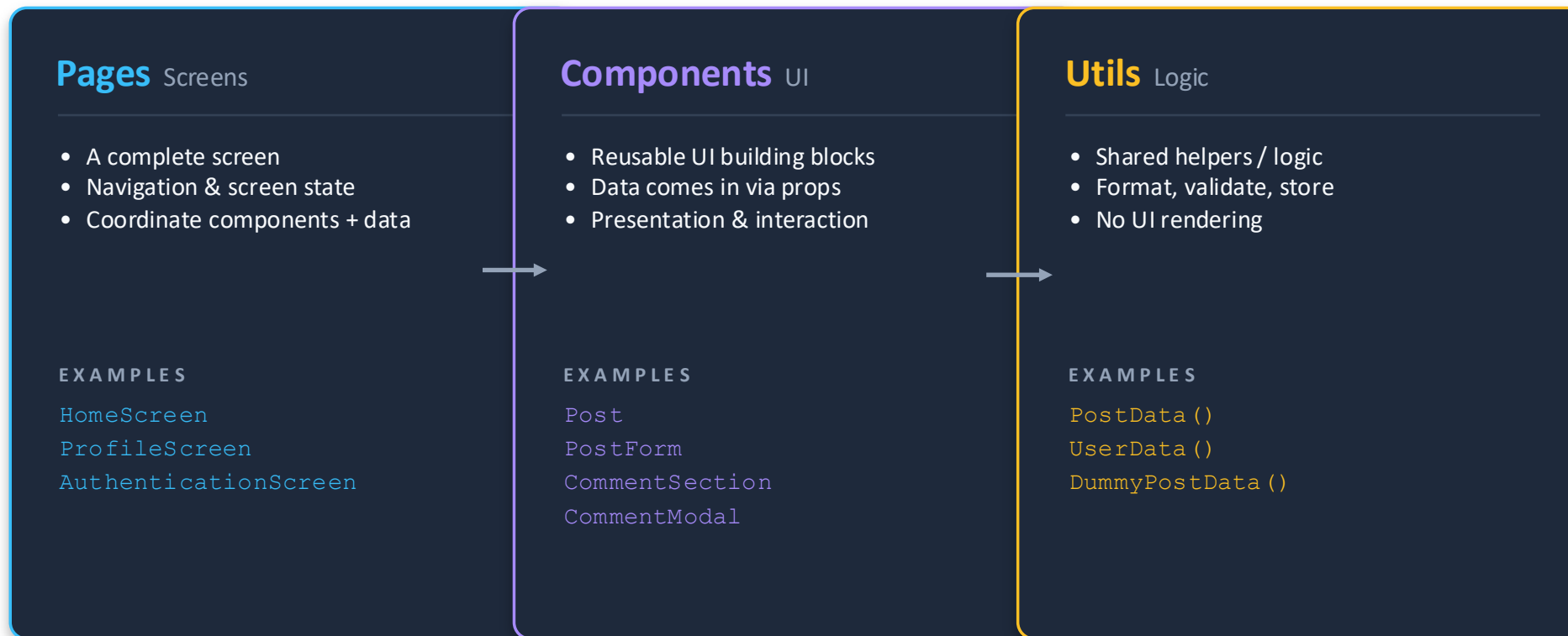
Create your first Firebase project

How a 2-hour session runs

0:00-0:05	Recap & objectives Where we are in the project; today's goals
0:05-0:45	Theory, Controlled forms & validation, Modal + PostForm TextInput, state, validation
0:45-1:00	Break —
1:00-1:40	Comments, Introduction to Firebase Create data without navigating away
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

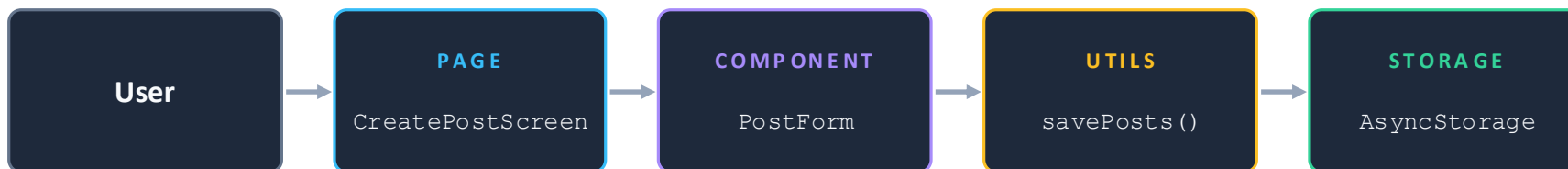
Designing a React Native app

- As an app grows, it is necessary to structure the app and split it into layers.



How the layers work together

- As an app grows, it is necessary to structure the app and split it into layers.



WHERE DOES IT GO?

Question	Place it in
Is it a screen?	Page
Is it reusable UI?	Component
Is it shared logic?	Utils
Does it handle navigation?	Page
Does it render UI?	Page / Component

THE GOAL

- Easier to maintain
- Easier to test
- Easier to extend
- Easier to collaborate on

A controlled, validated form

- value + onChangeText make an input controlled.
- Validate before saving; show a message on error.
- onClose / onSave callbacks talk to the parent.

```
export default function PostForm({ onSave, onClose }) {  
  const [title, setTitle] = useState('');  
  
  function onSubmit() {  
    if (!title.trim()) return alert('Title required');  
    onSave({ id: Date.now().toString(), title });  
    onClose();  
  }  
  
  return (  
    <View>  
      <TextInput value={title} onChangeText={setTitle}  
        placeholder="Title" />  
      <Pressable onPress={onSubmit}><Text>Add</Text></Pressable>  
    </View>  
  );  
}
```

A controlled, validated form

- Validate before submitting; show a message on error.

index.tsx

```
// Prepend the new post to state and persist the updated list to As
async function handleNewPost(post: PostData) {
  const updated = [post, ...posts];
  setPosts(updated);
  await AsyncStorage.setItem(POSTS_KEY, JSON.stringify(updated));
  Toast.show({ type: "success", text1: "Post saved!" });
}
```

```
<PostForm
  visible={formVisible}
  authorName={authorName}
  onClose={() => setFormVisible(false)}
  onSubmit={handleNewPost}
/>
```

components/PostForm.tsx

```
function handleSubmit() {
  if (!title.trim()) {
    setTitleError("Title is required.");
    return;
  }

  const newPost: PostData = {
    id: Date.now().toString(),
    title: title.trim(),
    description: description.trim(),
    // Join chips into a space-separated string: "#expo #reactnative"
    hashtags: chips.map((t) => `#${t}`).join(" "),
    author: authorName || "Anonymous",
    comments,
  };

  onSubmit(newPost);
  resetForm();
  onClose();
}
```

```
<View style={styles.header}>
  <TouchableOpacity onPress={handleClose}>
    <Text style={styles.cancelButton}>Cancel</Text>
  </TouchableOpacity>
  <Text style={styles.heading}>New Post</Text>
  <TouchableOpacity onPress={handleSubmit}>
    <Text style={styles.submitButton}>Post</Text>
  </TouchableOpacity>
</View>
```

THEORY • VALIDATION

Field Validation

- Validation checks whether user input satisfies business rules.
- Invalid data is rejected before it is stored.
- Frontend validation provides immediate feedback and avoid unnecessary requests.
- Frontend validation can not bypass backend validation for security reasons. Why? Because malicious user can bypass frontend app.
- We will introduce backend security rules in later sessions.

authenticationPage.tsx

```
function isValidEmail(email: string): boolean {
  return (
    email.includes("@") &&
    email.includes(".") &&
    !email.includes(" ")
  );
}

// Every validation problem is shown the same way: a red Toast at the top.
function showError(message: string) {
  Toast.show({ type: "error", text1: message });
}

export default function AuthenticationPage() {
  const [isSignUp, setIsSignUp] = useState(true);
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);

  // Empty every field. We call this after switching modes and after a...
  function clearForm() {
  }

  async function handleSignUp() {
    if (!name.trim()) {
      showError("Name is required.");
      return;
    }
    if (!isValidEmail(email.trim())) {
      showError("Please enter a valid email address.");
      return;
    }
    if (password.length < 6) {
      showError("Password must be at least 6 characters.");
      return;
    }
  }
}
```

Modal

- **A Modal shows content over the current screen and blocks the rest of the UI.**
 - Use for: confirmations, quick input forms, image previews, details.
 - Not navigation — the underlying screen stays mounted.
- **Key props:**
 - visible (boolean), animationType ('slide' | 'fade' | 'none'), transparent.
 - A React Native Modal does not control visibility by itself. Its visible prop must come from state somewhere.
 - onRequestClose() dismisses the Modal via pressing Android's back button, or swiping down an iOS pagesheet. It is recommended in cross-platform but not mandatory. For iOS it is optional.
 - Dismiss via a close button, tapping outside, or swipe-down (iOS). It can also be controlled by a custom Close or Cancel button.

Modal basics

```
const [open, setOpen] = useState(false);

<Pressable onPress={() => setOpen(true)}>
  <Text>New post</Text>
</Pressable>

<Modal visible={open} animationType="slide">
  <PostForm onClose={() => setOpen(false)} />
</Modal>
```

PostForm Modal

- State variable formVisible in index.tsx is used to control PostForm popup.
- onPress() and onClose() are used to setFormVisible for controlling the modal visibility through a state variable visible.
- Visible in PostForm is used to control visibility of Modal.

index

```
<TouchableOpacity style={styles.fab} onPress={() => setFormVisible(true)}>
  <Text style={styles.fabIcon}>+</Text>
</TouchableOpacity>

<PostForm
  visible={formVisible}
  authorName={authorName}
  onClose={() => setFormVisible(false)}
  onSubmit={handleNewPost}
/>
```

PostForm

```
export default function PostForm({ visible, authorName, onClose, onSubmit }:
PostFormProps) {
  function handleSubmit() {
    if (!title.trim()) {
      setTitleError("Title is required.");
      return;
    }
    const newPost: PostData = {
      id: Date.now().toString(),
      title: title.trim(),
      description: description.trim(),
      hashtags: chips.map((t) => `#${t}`).join(" "),
      author: authorName || "Anonymous",
      comments,
    };
    onSubmit(newPost);
    resetForm();
    onClose();
  }
  function handleClose() {
    resetForm();
    onClose();
  }
  return (
    <Modal
      visible={visible}
      animationType="slide"
      presentationStyle="pageSheet"
      onRequestClose={handleClose}
    />
  )
}
```

A Modal inside a modal

- The comment popup is a second `<Modal>` nested inside the `PostForm`'s `<Modal>`
- The inner `<Modal>` lives inside the outer modal so it layers on top of the sheet.
- Each modal requires its own `onClose()`.
- In `Postify`, we can add comments to existing posts and when creating a new post. It means that in both `PostForm` and `postDetails/[id]` we need comment function. A good coding practice is to extract the comment section and comment modal to independent components in order to reuse the code and reduce duplicity.

Comments Modal

- State variable *visible* is used to control the comment modal popup.
- Visible comes from modalVisible in CommentsSection, and controlled by onPress() and onClose().

CommentsSection

```

<TouchableOpacity
  style={styles.addButton}
  onPress={() => setModalVisible(true)}
>
  <Text style={styles.addText}>+ Add comment</Text>
</TouchableOpacity>

<CommentModal
  visible={modalVisible}
  onClose={() => setModalVisible(false)}
  onAdd={onAddComment}
/>

```

CommentsModal

```

3
4 type CommentModalProps = {
5   visible: boolean;
6   onClose: () => void;
7   // Called with the typed text when the user presses "Add".
8   onAdd: (text: string) => void;
9 };
10
11 // A small popup for typing one comment. It owns its own text state and
12 // clears itself whenever it closes, so the parent only handles the result.
13 export default function CommentModal({ visible, onClose, onAdd }: CommentModalProps) {
14   const [text, setText] = useState("");
15
16   > function handleAdd() { ...
17     }
18
19   > function handleCancel() { ...
20     }
21
22   return (
23     <Modal
24       visible={visible}
25       transparent
26       animationType="fade" (local function) handleCancel(): void
27       onRequestClose={handleCancel}
28     >
29       <View style={styles.backdrop}>
30         <KeyboardAvoidingView
31           behavior={Platform.OS === "ios" ? "padding" : "height"}
32         >
33           <View style={styles.dialog}>
34             <Text style={styles.title}>Add a comment</Text>
35             <TextInput
36               style={styles.input}
37               placeholder="Write your comment..."
38               value={text}
39               onChangeText={setText}
40               multiline

```

THEORY · COMMENTS

Adding a Comment

- Append (author, text) to a comments array, then render it with map()

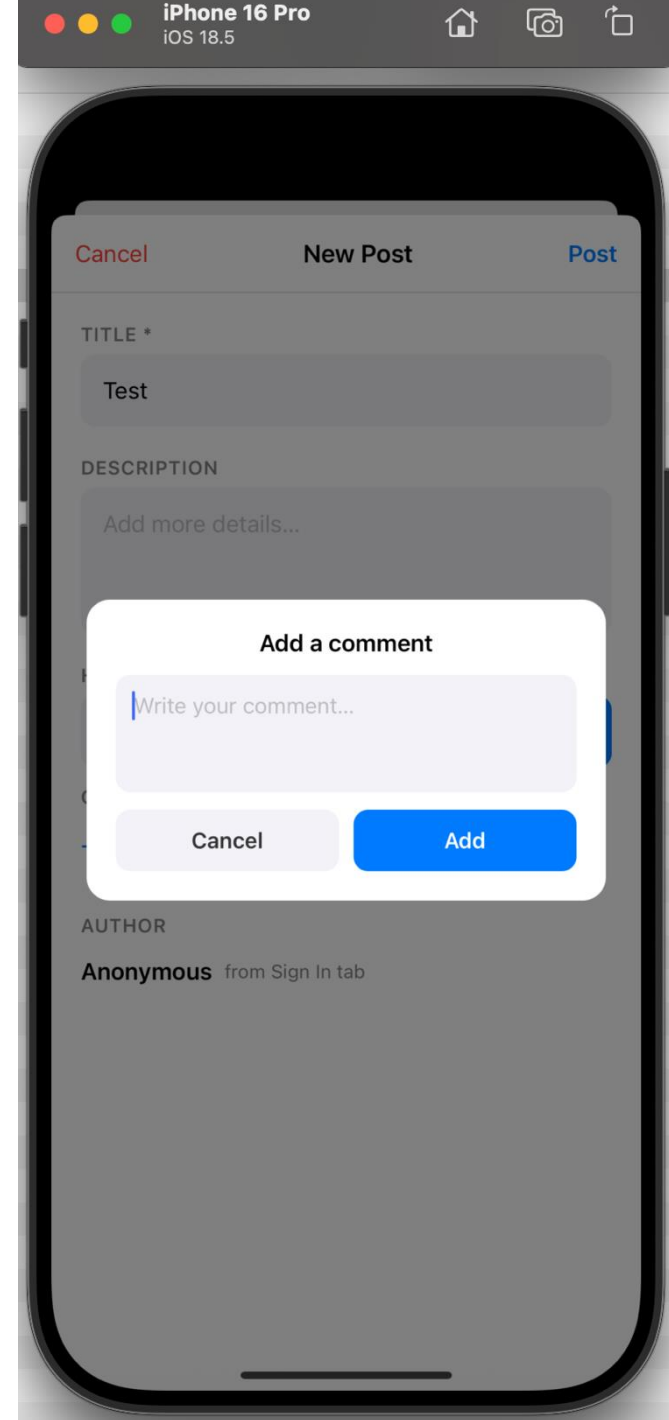
adding a comment

PostForm.tsx

```
const [comments, setComments] =
  useState<{ author: string; text: string }[]>([]);
function handleAddComment() {
  const text = commentInput.trim();
  if (!text) return;
  const author = authorName || "Anonymous";
  setComments((p) => [...p, { author, text }]);
  setCommentInput("");
  setCommentModalVisible(false);
}
```

THE FLOW

- Tap + Add comment → the popup modal opens
- Add runs handleAddComment
- Append { author, text } immutably with ...prev
- comments.map() renders each; author falls back to Anonymous



Adding a Comment

- We handle add comment differently in Postify.
- In postDetails/[id], existing post data is updated and stored back to AsyncStorage.

postDetails/[id]

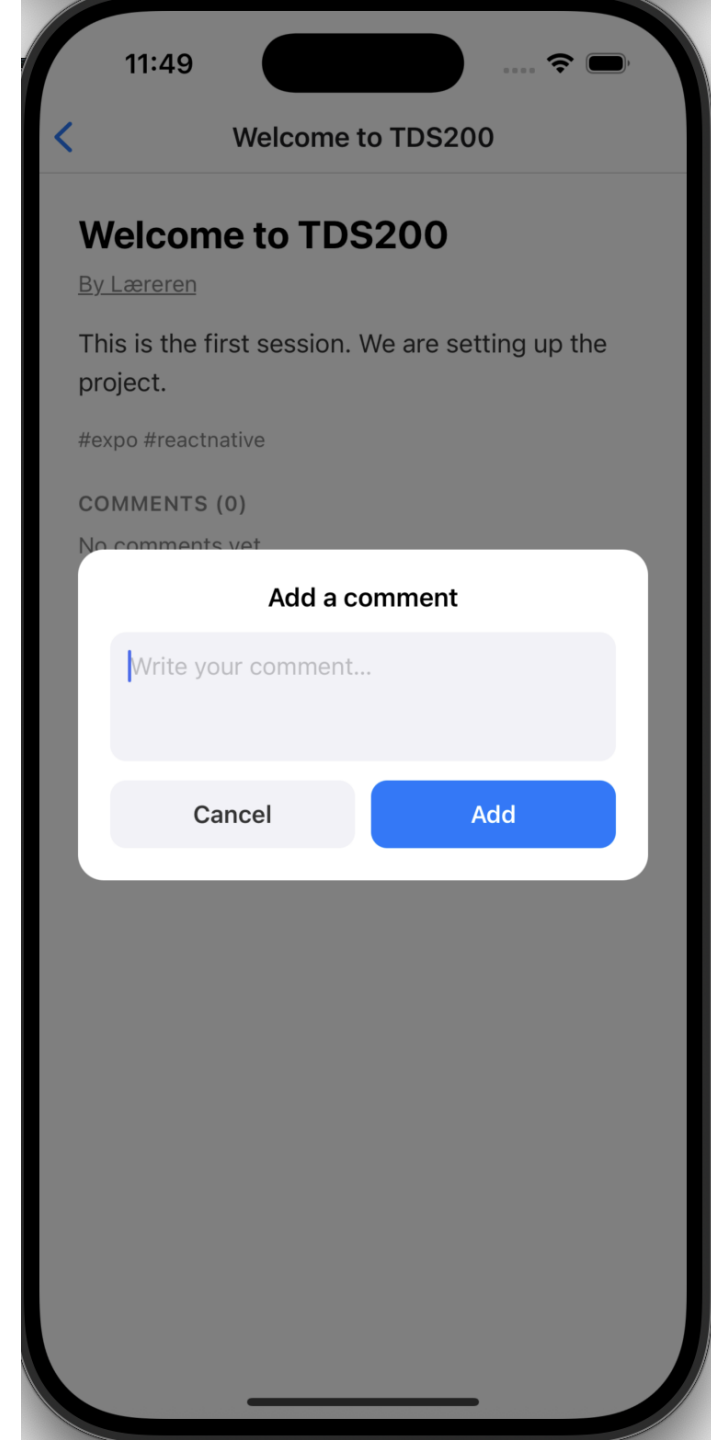
```
// Add the typed comment to this post, then save the whole posts list back.
async function handleAddComment(text: string) {
  if (!post) return;

  const newComment = { author: authorName || "Anonymous", text };
  const updatedPost: PostData = {
    ...post,
    comments: [...post.comments, newComment],
  };

  // Update the screen straight away.
  setPost(updatedPost);

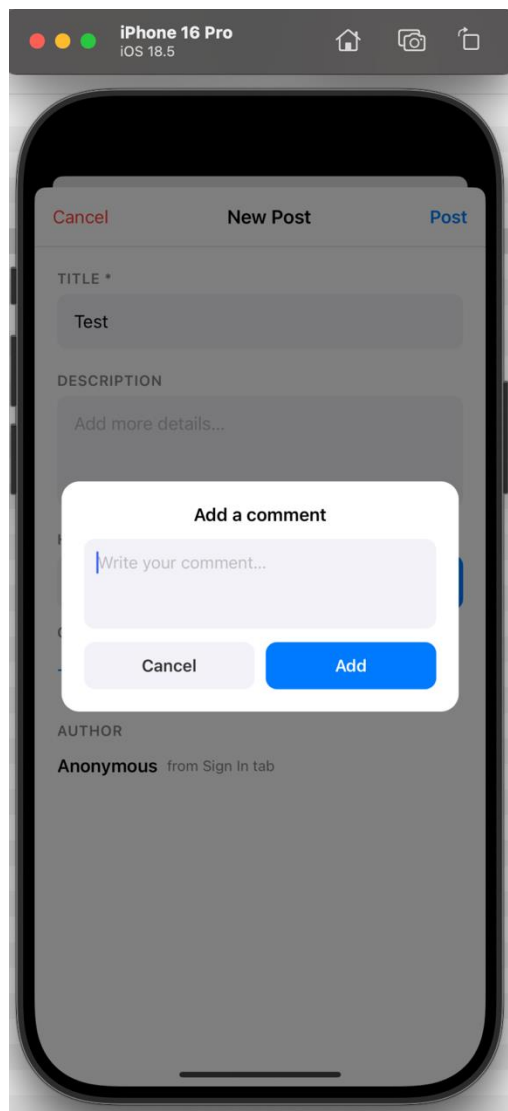
  // Persist: read the list, swap in the updated post, write it back.
  const stored = await AsyncStorage.getItem(POSTS_KEY);
  const posts: PostData[] = stored ? JSON.parse(stored) : [];
  const updatedPosts = posts.map((p) => (p.id === post.id ? updatedPost : p));
  await AsyncStorage.setItem(POSTS_KEY, JSON.stringify(updatedPosts));

  Toast.show({ type: "success", text1: "Comment added!" });
}
```



Adding a Comment

- In PostForm (when you create a new post), you update `newPost:PostData` with comments, and use `handleSubmit` to submit.
- It in terms calls `handleNewPost()` in `index.tsx`, and updates the posts array, then stores updated posts array back to `AsyncStorage`.



```
function handleSubmit() {
  if (!title.trim()) {
    setTitleError("Title is required.");
    return;
  }

  const newPost: PostData = {
    id: Date.now().toString(),
    title: title.trim(),
    description: description.trim(),
    // Join chips into a space-separated string: "#expo #reactnative"
    hashtags: chips.map((t) => `#${t}`).join(" "),
    author: authorName || "Anonymous",
    comments,
  };

  onSubmit(newPost);
  resetForm();
  onClose();
}

function handleClose() {
  resetForm();
  onClose();
}

function handleAddComment(text: string) {
  setComments((prev) => [
    ...prev,
    { author: authorName || "Anonymous", text },
  ]);
}
```

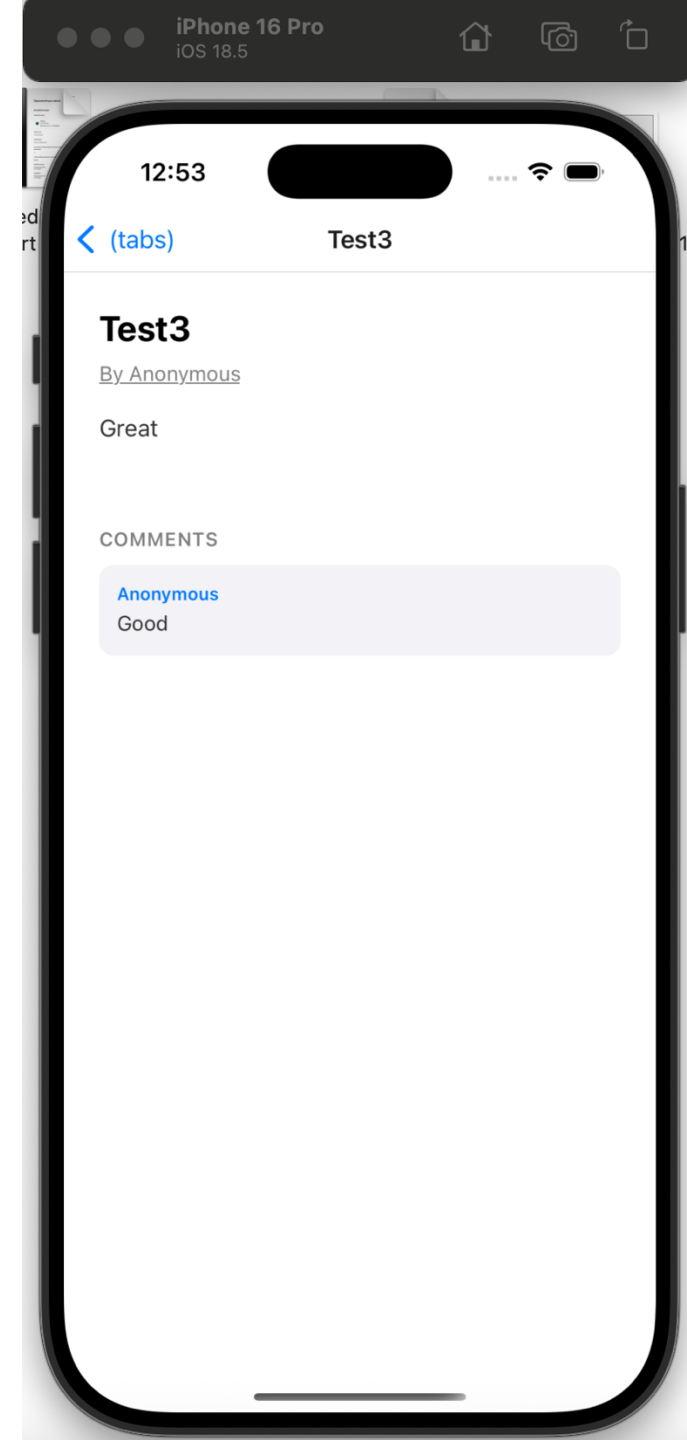
```
// Prepend the new post to state and persist the updated list to AsyncStorage.
async function handleNewPost(post: PostData) {
  const updated = [post, ...posts];
  setPosts(updated);
  await AsyncStorage.setItem(POSTS_KEY, JSON.stringify(updated));
  Toast.show({ type: "success", text1: "Post saved!" });
}
```

Show Comment

- Append (author, text) to a comments array, then render it with map()

display comments

```
{comments.map((comment, index) => (  
  <View key={index} style={styles.item}>  
    <Text style={styles.author}>{comment.author}</Text>  
    <Text style={styles.text}>{comment.text}</Text>  
  </View>  
))}  
  
{comments.length === 0 && (  
  <Text style={styles.noComments}>No comments yet.</Text>  
)}
```



THEORY • LOADING USER DATA

Loading Session User

- When the home tab regain focus, useEffect will loadAuthor, which fetches signed in user from AsyncStorage. (the same as profilePage...)
- authorName is then passed down to PostForm (child component) so that PostForm can use it when creating a new post.

```
// Re-run when the Home tab regains focus, so signing in on the
// Sign In tab updates the author straight away.
useFocusEffect(
  useCallback(() => {
    loadPosts();
    loadAuthor();
  }, [])
);
```

```
// Load the signed-in user's name so PostForm can show it as the author.
async function loadAuthor() {
  const stored = await AsyncStorage.getItem(SESSION_USER);
  if (stored) {
    const user: SessionUser = JSON.parse(stored);
    setAuthorName(user.name);
  } else {
    setAuthorName("");
  }
}
```

```
<PostForm
  visible={formVisible}
  authorName={authorName}
  onClose={() => setFormVisible(false)}
  onSubmit={handleNewPost}
/>
```

Hash Tag Array

- Map is used to display hashtag array.
- We have also field validation for hashtag.
- The hashtag array is stored in PostData as following:

```
// Join chips into a space-separated string: "#expo
#reactnative"
hashtags: chips.map((t) => `#${t}`).join(" "),
```

```
// Strip any leading # the user may have typed, then add to the chips array.
function handleAddChip() {
  const tag = hashtagInput.trim().replace(/^#+/, "");
  if (tag && !chips.includes(tag)) {
    setChips((prev) => [...prev, tag]);
  }
  setHashtagInput("");
}

function handleRemoveChip(tag: string) {
  setChips((prev) => prev.filter((t) => t !== tag));
}
```

```
/* Hashtags – chip input */
<Text style={styles.label}>Hashtags</Text>

/* Input row: text field + add button */
<View style={styles.hashtagRow}>
  <TextInput
    style={styles.hashtagInput}
    placeholder="machine learning"
    value={hashtagInput}
    onChangeText={setHashtagInput}
    onSubmitEditing={handleAddChip}
    autoCapitalize="none"
    returnKeyType="done"
  />
  <TouchableOpacity style={styles.addChipButton} onPress={handleAddChip}>
    <Text style={styles.addChipText}>+</Text>
  </TouchableOpacity>
</View>

/* Chips – shown below the input once at least one is added */
{chips.length > 0 && (
  <View style={styles.chipsContainer}>
    {chips.map((tag) => (
      <View key={tag} style={styles.chip}>
        <Text style={styles.chipText}>#{tag}</Text>
        <TouchableOpacity onPress={() => handleRemoveChip(tag)}>
          <Text style={styles.chipRemove}>x</Text>
        </TouchableOpacity>
      </View>
    ))}
  </View>
)}
```

Firebase: the default mobile backend

#1

Most widely deployed BaaS — ~4x
Supabase's footprint

Free

Spark plan: Auth + Firestore quotas at no
cost to start

50K

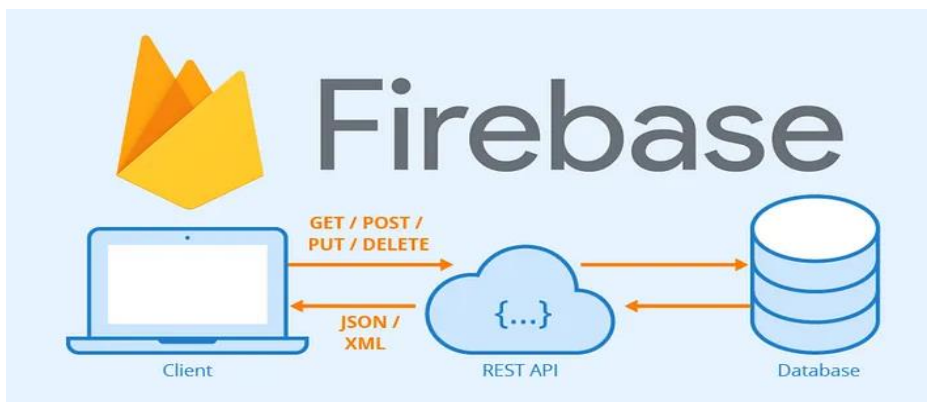
Firestore free daily reads (20K writes) on
Spark

\$2B

Supabase valuation (2025) — the main
open-source rival

- **SaaS → BaaS: you rent ready-made backend services.**

- Firebase is a cloud platform from Google
- It hosts services such as authentication, databases, file storage etc.
- It's categorized as SaaS - Software as a Service, or more specially:
BaaS - Backend as a Service
You pay to gain access to their software
- Popular as a backend for mobile applications



Firebase Products



Realtime Database

Build serverless apps by storing and syncing JSON data between your users in near-realtime, on or offline, with strong user-based security.

[Learn more](#)



Cloud Functions

Write and run app logic server-side without needing to set up your own server.

[Learn more](#)



Authentication

Add an end-to-end identity solution to your app for easy user authentication, sign-in, and onboarding in just a few lines of code.

[Learn more](#)



Cloud Messaging

Get infrastructure to reliably send and receive push messages between your server and devices, across platforms at no cost.

[Learn more](#)



Hosting

Deploy fast-loading, secure websites that are backed by a global CDN without all of the hassle.

[Learn more](#)



Cloud Storage

Store and serve user-generated content with ease as your app grows from prototype to production-ready.

[Learn more](#)

Local Storage vs Cloud Database

- **What is Local Storage?**

- Data is stored directly on user's device and can be accessed without an internet connection or user account. It is ideal for app settings, cached content, and temporary data that is specific to a single device.
- AsyncStorage and SecureStore are typical local storage.

- **What is Cloud Database?**

- Data is stored in Cloud database and can be shared across multiple devices and users, It is suitable for user generated content, synchronized application data, and information that should persist beyond a single device.
- Firebase provides two cloud database solutions: RealTime database and Cloud Firestore.

Collections & documents (NoSQL)

- **A Firestore database is a tree of collections and documents.**

- Collection ≈ a database table (e.g. 'posts').
- Document ≈ a row, but schema-free, so documents in the same collection can contain different fields.
- Documents can contain sub-collections (e.g. a post's 'comments').
- Firestore does not support traditional SQL JOIN operations — so data should be structured around how the application needs to read it.

shape of the data

```
posts (collection)
└─ <postId> (document)
    title: 'Sunset hike'
    images: ['https://...']
    createdAt: <timestamp>
    comments (sub-collection)
        └─ <commentId> { text }
```

Collections & documents (NoSQL)

- Both Realtime Database and Cloud Firestore are NoSQL Databases.

Both Realtime Database and Cloud Firestore are NoSQL Databases.

Realtime Database	Cloud Firestore
Stores data as one large JSON tree. • Simple data is very easy to store. • Complex, hierarchical data is harder to organize at scale. Learn more about the Realtime Database data model.	Stores data as collections of documents. • Simple data is easy to store in documents, which are very similar to JSON. • Complex, hierarchical data is easier to organize at scale, using subcollections within documents. • Requires less denormalization and data flattening. Learn more about the Cloud Firestore data model.

SETUP · FIREBASE

Create a Firebase project

1. Go to console.firebase.google.com and create a project.
2. Register an app (Web).
 - Web app here refers to the **Firebase JavaScript SDK** and **configuration type**, NOT the platform. The same Firebase web config can be used by your Expo Go app on both mobile platforms and web version.
3. Copy the config object and setup your Expo project.

terminal

```
npm install firebase
```

firebaseConfig.ts

```
import { initializeApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';

const app = initializeApp({
  apiKey: '...',
  authDomain: '...',
  projectId: '...',
  storageBucket: '...',
});

export const db = getFirestore(app);
```

Create a project - Step by step guidance

Step 1 - Create a Firebase project

- Go to Firebase console <https://console.firebase.google.com> (you need to login using your google account)
- Create a Firebase project.
- You can enable AI assistant (Gemini) and Google Analytics.



Create a new Firebase project

Integrate Firebase products to supercharge your app

× Create a project

Let's start with a name for
your project[?]

Project name

my-test-project

my-test-project-d2959

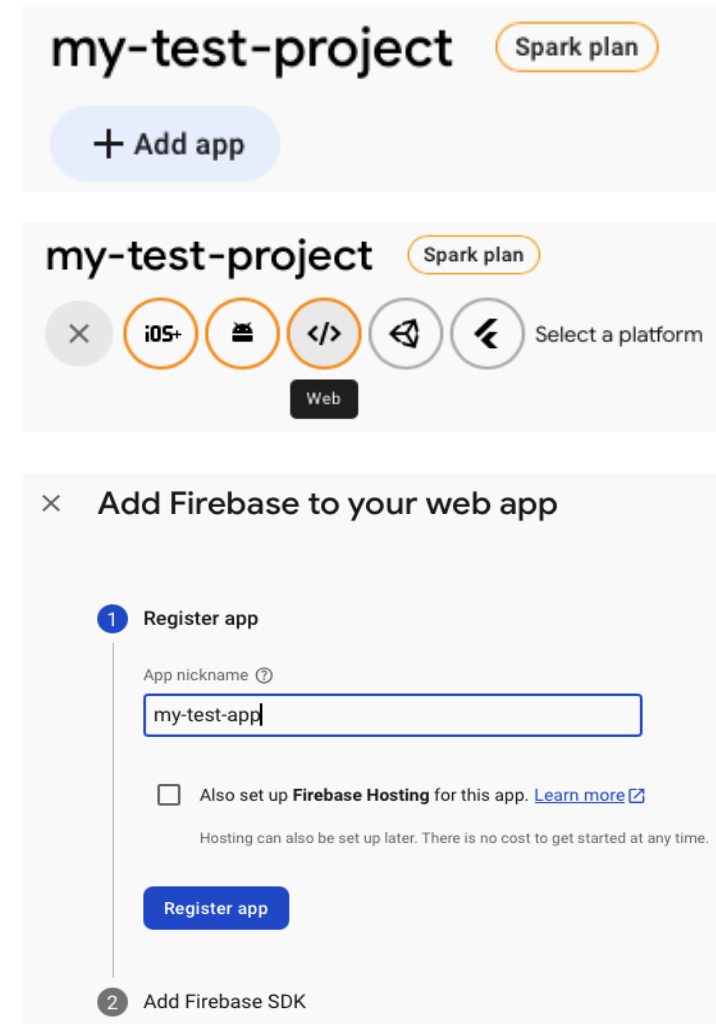
Create a project - Step by step guidance

Step 2 - Register an app

- A Firebase project is the backend container for all your services (Auth, Firestore, Storage, etc)
- When adding an app, Firebase will generate configuration keys, such as **apiKey**, **authDomain**, **projectId**, **storageBucket**., required by the selected SDK and platform.
- Multiple client apps can use the same Firebase project and share its backend services.
- Your client app would need these configuration keys to connect to Firebase.
- A Firebase app registration creates a Firebase identification and configuration for an existing client app. A Firebase app is NOT your client app!

You will get platform-specific setup by registering Firebase app.

- For Web and simulators, you get a JavaScript config object to use in your code.
- For Android, you download google-service.json
- For iOS, you download GoogleService-Info.plist.



Create a project - Step by step guidance

Step 2 – Continued

- Adding a Web app generates a configuration object for you to use in your client app.
- Then it generates a configuration object for you to use in your client app (Your Expo app).

✓

Register app

2

Add Firebase SDK

☒

Use npm

☐

Use a <script> tag

If you're already using [NPM](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([learn more](#)):

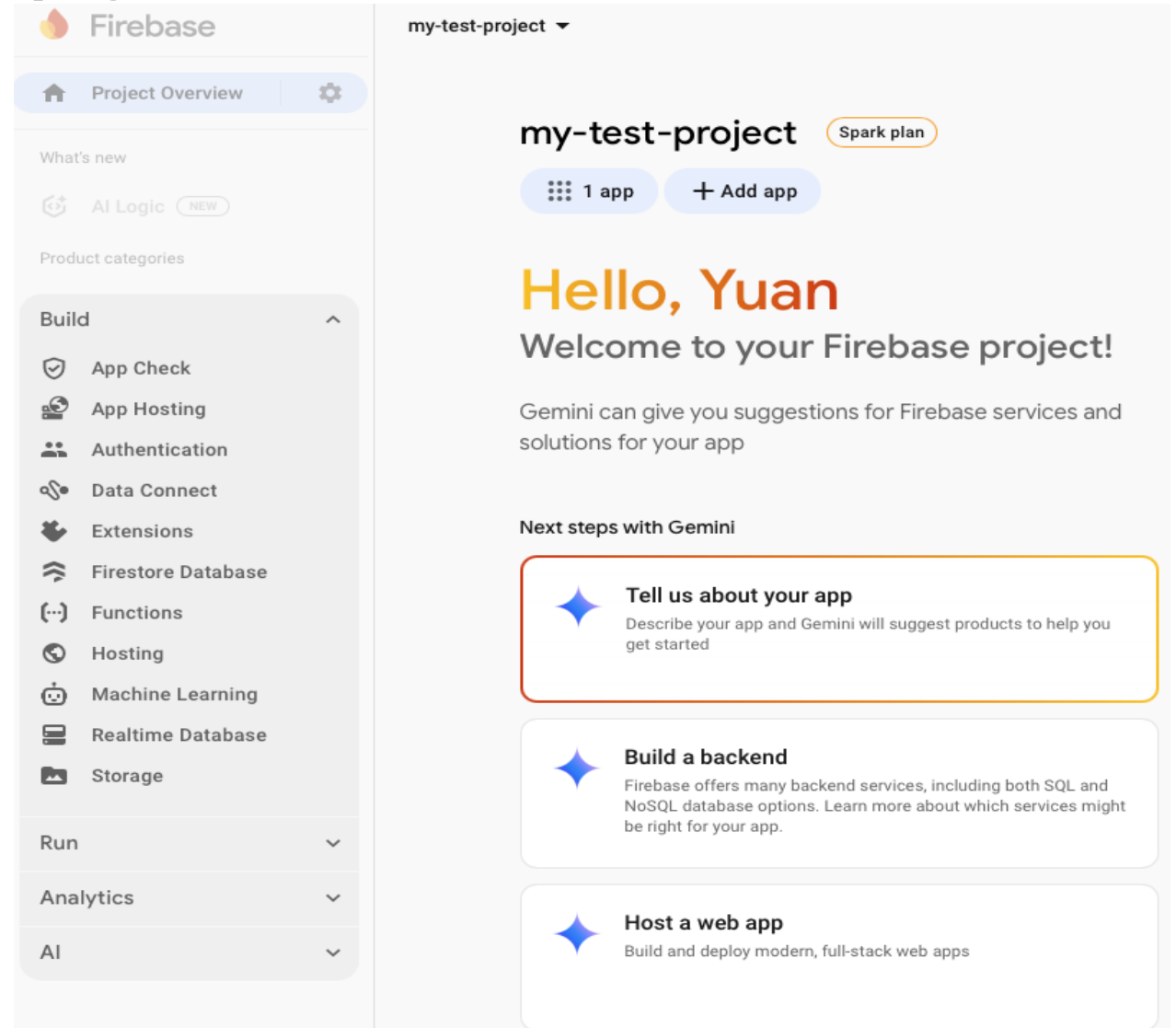
```
$ npm install firebase
```

Then, initialise Firebase and begin using the SDKs for the products that you'd like to use.

Create a project - Step by step guidance

Step 3 - Go to Firebase Console

- You can start configuring authentication, Firestore database, and Storage. (We will cover it in next session)
- Note that you can always go to Project overview/project settings to find the configuration object.
- From Firebase Console, you can also delete your app or remove the project completely.



Create a project - Step by step guidance

Step 4 – Configure client app

- You need to create `firebaseConfig.ts`, and import your firebase configuration generated when you created your firebase web app.
- A good coding practice is to store the secrets as keys in `.env`.
- Import those secrets from `.env` using `process.env.name`

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "xxxxxx",
  authDomain: "my-test-project-xxx.firebaseio.com",
  projectId: "my-test-project-xxx",
  storageBucket: "my-test-project-xxx.firebaseio.com",
  messagingSenderId: "xxxx",
  appId: "xxxx",
  measurementId: "xxx"
};
// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);
```

ASSIGNMENT 5

Create posts & comments



YOUR TASKS

- Add a Delete button to each post card from FlatList on the Home Page. When clicked, a confirmation Modal popup appears (Delete this post?). If the user confirms, the post is removed from AsyncStorage and disappears from the FlatList.

DELIVERABLE

A post feed with delete functionality, including a confirmation modal popup and persistent removal of posts from AsyncStorage.

EST. EFFORT

≈ 1-3 hours